Excel Dataset: Data Cleaning Using Python and pandas.

---

1. Import Libraries and Load Dataset

In [1]:
```
import pandas as pd
import numpy as np
```

In [2]:
```
# Load Excel File
file_name = "Dataset for Data Analytics.xlsx"
df = pd.read_excel(file_name)
```

In [3]:
```
# Display first few row
df.head()
```

Out[3]:

	OrderID	Date	CustomerID	Product	Quantity	UnitPrice	ShippingAddress	PaymentMethod	OrderStatus	TrackingNumber	Items
0	ORD200000	2023-01-04	C72649	Monitor	5	570.62	928 Main St	Debit Card	Shipped	TRK37947903	
1	ORD200001	2024-08-23	C75739	Phone	2	151.35	823 Main St	Online	Shipped	TRK91186779	
2	ORD200002	2024-02-27	C81728	Tablet	5	550.68	512 Main St	Credit Card	Cancelled	TRK42903982	
3	ORD200003	2023-10-15	C33540	Chair	1	273.19	275 Main St	Debit Card	Returned	TRK62788070	
4	ORD200004	2025-05-08	C81840	Printer	4	626.01	668 Main St	Online	Delivered	TRK29241424	

In [4]:
```
# Check dataset Shape
print("Rows and Columns:",df.shape)
```

```
Rows and Columns: (1200, 14)
```

Explanation:

- `pandas` is used for data manipulation.
- `numpy` helps with numerical operations and missing values.
- `read_excel()` loads the Excel dataset into a DataFrame.
- `head()` Previews the data.
- `shape` shows the number of rows and columns.

---

2. Initial Dataset Inspection

In [5]:
```
#View Column names
df.columns
```

Out[5]:
```
Index(['OrderID', 'Date', 'CustomerID', 'Product', 'Quantity', 'UnitPrice',
       'ShippingAddress', 'PaymentMethod', 'OrderStatus', 'TrackingNumber',
       'ItemsInCart', 'CouponCode', 'ReferralSource', 'TotalPrice'],
      dtype='object')
```

In [6]:
```
# Check data types and missing values
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 1200 entries, 0 to 1199
Data columns (total 14 columns):
 #   Column           Non-Null Count  Dtype
---  ------           --------------  -----
 0   OrderID          1200 non-null   object
 1   Date             1200 non-null   datetime64[ns]
 2   CustomerID       1200 non-null   object
 3   Product          1200 non-null   object
 4   Quantity         1200 non-null   int64
 5   UnitPrice        1200 non-null   float64
 6   ShippingAddress  1200 non-null   object
 7   PaymentMethod    1200 non-null   object
 8   OrderStatus      1200 non-null   object
 9   TrackingNumber   1200 non-null   object
 10  ItemsInCart      1200 non-null   int64
 11  CouponCode       891 non-null    object
 12  ReferralSource   1200 non-null   object
 13  TotalPrice       1200 non-null   float64
dtypes: datetime64[ns](1), float64(2), int64(2), object(9)
memory usage: 131.4+ KB
```

In [7]:
```
# Count missing values in each column
df.isnull().sum()
```

Out[7]:
```
OrderID             0
Date                0
CustomerID          0
Product             0
Quantity            0
UnitPrice           0
ShippingAddress     0
PaymentMethod       0
OrderStatus         0
TrackingNumber      0
ItemsInCart         0
CouponCode        309
ReferralSource      0
TotalPrice          0
dtype: int64
```

Explanation:

This step helps identify:

- Missing values
- Incorrect data types

Observation:

The main missing-value issue appears in `CouponCode` column, with `309` missing values out of `1200` total rows

---

3. Investigating `CouponCode` Column

In [8]:
```
# Understanding Existing Values
df['CouponCode'].value_counts(dropna=False)
```

Out[8]:
```
CouponCode
FREESHIP    313
NaN         309
WINTER15    292
SAVE10      286
Name: count, dtype: int64
```

Explanation:

This helps determine:

- Whether missing values are meaningful
- Frequency of coupon usage
- Whether a coupon was optional

In [9]:
```
# Handling Missing CouponCode Values
df['CouponCode'] = df['CouponCode'].fillna('No Coupon')
```

```
# Verify Results
df['CouponCode'].isnull().sum()
```

Out[9]: 0

Explanation:

Why this approach? By understanding the existing values:

- Missing values indicate “no discount used”
- Therefore, Customers were allowed to purchase without a coupon
- And also, You want to preserve all sales records to prevent unnecessary data loss

In [10]:
```
# Check CouponCode Formatting
df['CouponCode'].unique()
```

Out[10]: array(['SAVE10', 'FREESHIP', 'No Coupon', 'WINTER15'], dtype=object)

---

4. Check for Duplicate Records

In [11]:
```
# Count dupliacte rows
duplicates = df.duplicated().sum()
print("Duplicate Rows:", duplicates)
```

```
Duplicate Rows: 0
```

---

5. Validate Data Types

In [12]:
```
# Check Current Data Types
df.dtypes
```

Out[12]:
```
OrderID                  object
Date             datetime64[ns]
CustomerID               object
Product                  object
Quantity                  int64
UnitPrice                float64
ShippingAddress          object
PaymentMethod            object
OrderStatus              object
TrackingNumber           object
ItemsInCart               int64
CouponCode               object
ReferralSource           object
TotalPrice               float64
dtype: object
```

Observation:

All columns values are consistent with their respective data types, making the dataset suitable for the next step of analysis

---

6. Inspecting for Invalid Numeric Values

Detect the possibility of negative or impossible values. Why?

These may indicate:

- Data-entry errors
- Refund transactions

In [13]:
```
# Check for row with either negative quantities or invalid prices
invalid_row_count = df[(df['Quantity'] < 0) | (df['UnitPrice'] <= 0)].shape[0]

print(f"Invalid Rows:{invalid_row_count}")
```

```
Invalid Rows:0
```

Observation:

There are no records containing negative quantities or invalid price values.

---

6. Validate TotalPrice Consistency

Why?

This detects:

- Calculation errors
- Manual editing mistakes

In [14]:
```
# Create expected total
expected_total = (df['Quantity'] * df['UnitPrice'])

# Compare against TotalPrice
invalid_total = df[abs(df['TotalPrice']- expected_total) > 0.01]

print("Rows with Inconsistent totals:",invalid_total.shape[0])
```

```
Rows with Inconsistent totals: 0
```

6. Standardize Text Columns

Why?

This prevents inconsistencies like `Delivered`, `DELIVERED`, `delivered` being treated as different categories.

In [15]:
```
text_columns = ['Product','PaymentMethod','OrderStatus','ReferralSource']

for col in text_columns:
    df[col] = df[col].astype(str).str.strip().str.title()
```

---

7. Checking for Unique Values for Categorical Columns

This helps identify:

- Typographical errors
- Unexpected categories
- And inconsistent labels

In [16]:
```
categorical_columns = ['Product','PaymentMethod','OrderStatus','ReferralSource', 'CouponCode']

for col in categorical_columns:
    print(f"\n{col}")
    print(df[col].unique())
```

```
Product
['Monitor' 'Phone' 'Tablet' 'Chair' 'Printer' 'Laptop' 'Desk']

PaymentMethod
['Debit Card' 'Online' 'Credit Card' 'Gift Card' 'Cash']

OrderStatus
['Shipped' 'Cancelled' 'Returned' 'Delivered' 'Pending']

ReferralSource
['Instagram' 'Referral' 'Email' 'Facebook' 'Google']

CouponCode
['SAVE10' 'FREESHIP' 'No Coupon' 'WINTER15']
```

---

8. Detect Outliers

Outliers may affect:

- Revenue Analysis
- Forecasting

In [17]:
```
# Detect Outliers in UnitPrice
Q1 = df['UnitPrice'].quantile(0.25)
Q3 = df['UnitPrice'].quantile(0.75)
IQR = Q3-Q1
```

```
lower_bound = Q1 - 1.5*IQR
upper_bound = Q3 + 1.5*IQR

outlier = df[(df['UnitPrice'] < lower_bound) | (df['UnitPrice'] > upper_bound)]

outlier.shape[0]
```

Out[17]:
```
0
```

---

9. Final Missing Value Audit

To ensure:

- Important columns are complete
- No intended missing value remains

In [18]:
```
df.isnull().sum()
```

Out[18]:
```
OrderID           0
Date              0
CustomerID        0
Product           0
Quantity          0
UnitPrice         0
ShippingAddress   0
PaymentMethod     0
OrderStatus       0
TrackingNumber    0
ItemsInCart       0
CouponCode        0
ReferralSource    0
TotalPrice        0
dtype: int64
```

---

10. Save the Cleaned Dataset

In [19]:
```
# Save cleaned dataset
final_file = "Cleaned_Dataset.xlsx"
df.to_excel(final_file, index=False)
print("Cleaned dataset saved successfully.")
```

```
Cleaned dataset saved successfully.
```

Conclusion:

The dataset **`Dataset for Data Analytics.xlsx`** is largely clean and well-structured, with minimal data quality issues identified during the assessment. The primary observation relates to missing values in the **`CouponCode`** column, which are likely meaningful rather than erroneous. These missing entries plausibly represent transactions where no discount was applied, as suggested by the consistency of the remaining coupon patterns.

Overall, the dataset demonstrates good data integrity, with key validation checks confirming the absence of critical issues such as invalid prices or negative quantities. The data preparation process has therefore focused on ensuring accuracy, consistency, and completeness prior to analysis in order to prevent biased or skewed insights during exploratory data analysis.
